

Highlights

Blockchain access control for agricultural edge IoT: a 61-day Hyperledger Fabric field study and implementation postmortem

Forcha Glen Beloa, Michael Ekonde Sone, Elie Fute Tagne, André Paul Mvogo Asse

- A 61-day deployment measured blockchain access control on agricultural edge IoT.
- Recorded authorization timing varied by 3.4 ms across four tested peer counts.
- Latency fell 399 ms from 4 to 32 configured Fabric peers; throughput was not tested.
- An audit found an unpatched policy defect and two fixed transport/signature bugs.
- Missing episode keys blocked end-to-end revocation-exposure analysis.

Blockchain access control for agricultural edge IoT: a 61-day Hyperledger Fabric field study and implementation postmortem^{*}

Forcha Glen Beloa^{a,*}, Michael Ekonde Sone^a, Elie Fute Tagne^a and André Paul Mvogo Asse^b

^aDepartment of Computer Engineering, Faculty of Engineering and Technology, University of Buea, Buea, Cameroon

^bÉcole Nationale Supérieure Polytechnique de Douala, Université de Douala, Douala, Cameroon

ARTICLE INFO

Keywords:

Hierarchical role-based access control

Hyperledger Fabric

Agricultural IoT

Edge computing

Performance measurement

Certificate revocation

Reproducible research

ABSTRACT

Blockchain access control for the Internet of Things is usually evaluated by quoting a peak transaction rate and a mean end-to-end latency, which says nothing about how much of that latency the access-control layer is responsible for. We report a 61-day deployment of a hierarchical role-based access control (HRBAC) framework on Hyperledger Fabric 2.5, running on a 2 ha farm in Sfax, Tunisia with 50 sensors, 4 Raspberry Pi 4B gateways and 4 zones, over which 209 000 authorization decisions were recorded. The chaincode logged a timing field, `rbac_overhead_ms`, on every write; because its exact instrumentation boundaries cannot be reconstructed from the deployed source, we treat it as an authorization-associated implementation span rather than a directly identified permission-evaluation cost, and it averages 319 ms, 26.2% of the 1 220 ms write path. Using configured logical endorsing-peer count as an experimental lever rather than as a source of Fabric stage attribution, the remaining latency fell by 397 ms across an eightfold increase from 4 to 32 peers, leaving 503 ms unaccounted for at the largest configuration tested; neither quantity is an independently measured platform component. The recorded span itself is the term that held still: a block-bootstrap comparison of the extreme configurations puts the difference at 2.58 ms, inside a ± 10 ms margin fixed before this reanalysis, and it costs a near-constant 9.9% of throughput across a tenfold concurrency range.

The second half of the paper is a postmortem, and we think it is the more transferable contribution. Auditing the implementation against the manuscript uncovered three defects that ran undetected for the full deployment: the role hierarchy inherited audit access upward, so the auditor separation the design exists to provide was never in force, and it remains unpatched in the released artifact so as not to invalidate the deployment these measurements describe; the residue transport was parameterised outside its own recovery bound, so every one of the readings reassembled from two of three packets decoded incorrectly; and the gateway verified signatures over a different message than the sensor signed, a fail-open default that let unsigned readings pass. The latter two are fixed in the released code, but the corrected implementation's performance has not been separately measured, so equivalence with the reported timings is not established. None of the three would have been visible to a benchmark that ran for an afternoon. We report what the deployment measured and what it did not establish, and release the traces and the pipeline that regenerates every number here.

1. Introduction

A precision farm is a multi-stakeholder system before it is a technical one. Soil probes, weather stations and irrigation actuators produce readings continuously [52], and the people who consume those readings want different and partly conflicting things from them. Farmers need operational control. Agronomists need history but no actuation rights. Certification bodies auditing organic or GlobalGAP compliance need evidence of

^{*} The trace dataset, chaincode, firmware, formal specification and the analysis pipeline that regenerates every number, table and figure in this article are available at <https://github.com/forchag/blockchain-fet>.

^{*}Corresponding author

forchag@cam.ac.cm (F.G. Beloa); ekonde.sone@gmail.com (M.E. Sone); eliefute@yahoo.fr (E.F. Tagne); mvogoasseandre@gmail.com (A.P.M. Asse)
ORCID(s): 0009-0003-1972-6100 (F.G. Beloa)

practice without visibility into the agronomic detail a farm treats as commercially sensitive. Supply-chain partners need provenance but not real-time operations. Regulators need environmental records.

Handing this authorization matrix to a single server concentrates the risk in one place. An attacker who takes that server gains not only the data but, on a farm with networked valves, the irrigation itself [6], and the audit log that would reconstruct the incident is written by the machine that was compromised. Surveying agricultural IoT deployments, Ferrag et al. [10] found most lacked fine-grained access control at all. Kamilaris et al. [18] showed that blockchain work in agriculture had settled almost entirely on post-harvest supply-chain tracking, and recent surveys of IoT, AI and blockchain convergence in agriculture reach the same conclusion about pre-harvest authorization [33]. Putting role assignments, revocations and access decisions on a permissioned ledger removes the single point of trust. That much is settled.

What is not settled is the cost, and specifically where the cost falls.

1.1. Why the agricultural edge context matters

The constraints in Section 3 are not incidental to the measurement; they are why the deployment answers questions a laboratory benchmark cannot. Sensor nodes run on constrained energy budgets that make a persistent policy-decision-point connection unattractive, so the design pushes caching and enrollment logic to the edge gateway rather than assuming a reachable server. Connectivity in rural Tunisia is intermittent by the ordinary standard of the site, not by injected fault, so the revocation and outage behaviour in Sections 6.5 and 6.8 reflects a failure mode a two-day lab trial would not encounter: a certificate revoked while its gateway is disconnected. LoRa's low bit rate is why residue-coded transmission (Section 4.2) exists at all, and why the defect in Section 7.2 could survive undetected. Physical exposure means keys live in commodity microcontroller eFuse rather than a hardware security module, which sets the honest bound on what the signature scheme in Section 7.3 could have protected even had it been implemented correctly. None of this is unique to agriculture, but the combination, long-running, energy-constrained, intermittently connected, physically exposed, is what makes a multi-week field deployment a different instrument than a data-centre benchmark of the same chaincode.

1.2. The measurement gap

Among the systems surveyed in Table 1, evaluation is typically a laboratory or simulated run of hours to days, reported as a peak throughput and a mean latency. Both aggregate over a pipeline with at least four separable stages, so an engineer told that a transaction takes 1220ms cannot tell whether to rewrite the policy engine, restructure endorsement, replace the ordering service, or accept the figure as a consensus floor. Existing evaluations rarely combine long-duration field observations with an implementation-level authorization timing span, controlled configuration experiments and a post-deployment audit; that combination, not any single measurement, is the gap this paper addresses.

Attributing time to Fabric's internal stages needs one transaction timestamped at each hand-off, which our instrumentation does not do. A coarser accounting is available without it. The chaincode logs a timing field on every write; because its own start and stop boundaries were not documented at deployment time, we treat it as an authorization-associated span rather than a stage measurement, and endorsing-peer count acts as a lever on what remains. What that cannot do is split the remainder into ordering, validation and commit, which stay aggregated throughout (Section 8.3).

The second half of the paper reports something the measurements did not produce. Auditing the implementation against its own specification turned up three defects that had run for the whole deployment, none of which any benchmark would surface, and we think that pattern generalises further than the timings do.

Abang et al. [1], reviewing latency modelling for Hyperledger Fabric from an IoT perspective, make a closely related point. The literature models latency in the abstract but rarely connects those models to the behaviour of a specific deployed use case. They decompose Fabric latency into endorsement, ordering and validation phases and identify block size, endorsement policy, peer count and state database as the parameters that govern it. What has been missing is a deployment that measures those phases separately, under a real workload, for long enough that the slow-moving parts of the system have a chance to misbehave. We do not supply that stage-level measurement either; what we add is a long-duration field record against which the recorded span holds up, plus the audit that a short benchmark would not have prompted.

The intuitive expectation turns out to be wrong, which is the main reason we think the measurement is worth reporting. Access control is the component under study, so it is natural to assume it dominates the cost. The recorded span does not: it is a modest and remarkably stable fraction of the write path, and the larger, more variable share sits in the layer that access-control papers usually treat as background infrastructure.

A second gap is temporal. Certificate lifecycle management, revocation propagation and enrollment recovery are the mechanisms most likely to fail in the field and least likely to be exercised by an afternoon of benchmarking. A revocation pipeline that behaves perfectly across fifty laboratory trials can still show a materially different delay distribution across two months of intermittent rural connectivity. Reporting only the happy path understates operational risk in a way that matters to anyone deciding whether to deploy.

These questions structure the rest of the paper:

- RQ1** What latency and throughput differences are observed between the tested access-control-enabled configuration and the access-control-free comparison configuration?
- RQ2** How is configured logical peer count associated with client-observed write latency under the deployed topology?
- RQ3** Which implementation and observability failures survived the field deployment, and why did the existing tests not detect them?

1.3. Contributions

We built an HRBAC framework on Hyperledger Fabric 2.5 for the constraints of agricultural edge deployment, ran it on a working farm for 61 days, and instrumented the authorization-associated span and the end-to-end transaction path.

1. **A 61-day field characterisation.** An access-control-enabled agricultural edge deployment recording 209 000 authorization decisions over 50 sensors, 4 gateways and 4 zones, addressing RQ1 (Section 6).
2. **A controlled comparison across tested configurations.** The recorded authorization-associated span averages 319 ms, 26.2% of the 1 220 ms write path, and a block-bootstrap comparison of the 4- and 32-peer configurations finds it changes by only 2.58 ms while the remaining latency falls by 397 ms, addressing RQ2. Neither the span's internal boundaries nor the remainder's internal stages are separately identified (Sections 6.1 and 6.4).
3. **An implementation postmortem.** An audit of the implementation against its own specification found three defects that survived 61 days of operation, plus one observability gap, addressing RQ3. Each is a failure mode that an afternoon of benchmarking cannot surface, and together they are the part of this work most likely to transfer to other deployments (Section 7).
4. **A reproducible artifact and cross-layer testing recommendations.** The trace dataset, chaincode, firmware, formal specification and the analysis pipeline that regenerates every number, table and figure here are released under a tagged version (Section 5.5), together with the seam-level checks the postmortem motivates: a permission table generated from the deployed policy code rather than transcribed, an encoder test pinned to its recovery bound, and a verifier test pinned to the firmware's exact byte layout (Section 7.5).

Section 2 positions the work. Section 3 presents the access-control model and the enrollment protocol, Section 4 the implementation, Section 5 the measurement methodology, Section 6 the results, Section 8 what follows from them for system design, and Section 9 concludes.

2. Related work

2.1. Access control for constrained devices

Role-based access control as formalised by Sandhu et al. [36] and standardised by NIST into Core, Hierarchical and Separation-of-Duty variants [11] is still the reference model for multi-stakeholder authorization. Its IoT adaptations share a structural assumption that becomes awkward in the field: some trusted decision component has to be reachable at the moment access is requested. RBAC extensions for IoT

Table 1

Evaluation methodology in representative blockchain access-control systems. *Auth. span timed* indicates whether the authorization decision is timed separately from the rest of the write path; none of the systems below, including this one, separates ordering, validation and commit from each other. N/R, not reported.

System	Platform	Setting	Duration	Auth. span timed	Revocation measured
FairAccess [29]	Bitcoin	Analytical	N/A	No	No
BlendCAC [48]	Ethereum	Testbed	Hours	No	No
Novo [27]	Ethereum	Testbed	Hours	No	No
Shih et al. [39]	Fabric	Lab (ARM64)	Days	No	No
Zaidi et al. [51]	Fabric	Simulation	N/A	No	No
Pajooh et al. [30]	Fabric	Lab	N/R	No	No
Putra et al. [31]	Fabric	Lab	N/R	No	No
Usman et al. [44]	Fabric	Cloud	Days	No	No
González et al. [13]	Fabric	Lab	N/R	No	No
Idrissi & Palmieri [17]	Fabric	Lab (ARM)	N/R	Partial	No
Bandara et al. [5]	Fabric	Simulation	N/A	No	No
This work	Fabric	Field	61 days	Yes	Yes

[49], attribute-based evaluation [15, 38, 37] and capability-token delegation [14] place that trust in a policy engine, an attribute authority and a credential issuer respectively. Sciancalepore et al. [37] quantify what open-standard authorization stacks cost on constrained hardware, which is the empirical case for moving policy evaluation off the sensor entirely.

Each of these reduces part of the problem and keeps a central dependency somewhere. Ye et al. [49] and Servos and Osborn [38] retain a central enforcement point. Hernández-Ramos et al. [14] move authorization into tokens, which distributes revocation instead of solving it. Alshehri and Sandhu [3] split the model, giving RBAC to humans and capability-based control to devices, and the role store stays centralised. Novo [27] argued early that the role store itself belongs on a distributed ledger. We take that as given and ask what it costs.

2.2. Blockchain-based access control and how it is evaluated

Capability-token systems on public chains established feasibility without practicality. FairAccess [29] encodes grants as Bitcoin-like transfers and inherits confirmation windows measured in minutes. BlendCAC [48] uses ERC-721 tokens and incurs per-transaction fees that no sensor duty cycle can absorb. Neither addresses role hierarchy, geographic scoping or seasonal expiry, which is what an agricultural deployment actually needs.

Permissioned platforms remove the latency and fee obstacles, and Fabric-based IoT work has grown accordingly: testbed architectures and performance models [39, 30, 19], trust and reputation layers [31], contract-enforced domain policy on cloud-hosted nodes where memory and thermal limits do not apply [44], and a data marketplace at scale [13]. All of these report aggregate throughput and latency from a controlled setting. Two are closer to our method. Idrissi and Palmieri [17] give an end-to-end Fabric latency breakdown on constrained ARM hardware in a healthcare context, which is the nearest methodological precedent for the partition in Section 6.1.1. Tzenetopoulos et al. [43] establish a Raspberry Pi 3B+ reference point we compare against in Section 8.2.

Table 1 summarises how these systems are evaluated. The pattern is consistent: laboratory testbeds or simulation, runs of hours or days, aggregate reporting. We have not found prior work that times an authorization-associated span separately from the rest of the write path on a field deployment of this duration, or that reports revocation behaviour over a multi-week window. That combination, rather than any single mechanism in Section 3 or a validated stage-by-stage decomposition, is the contribution.

2.3. Agricultural IoT and edge infrastructure

Blockchain in agriculture has concentrated on traceability after harvest [42, 22, 5], where the unit of interest is a shipment rather than a zone-scoped actuation. Safeer et al. [34] combine computer vision,

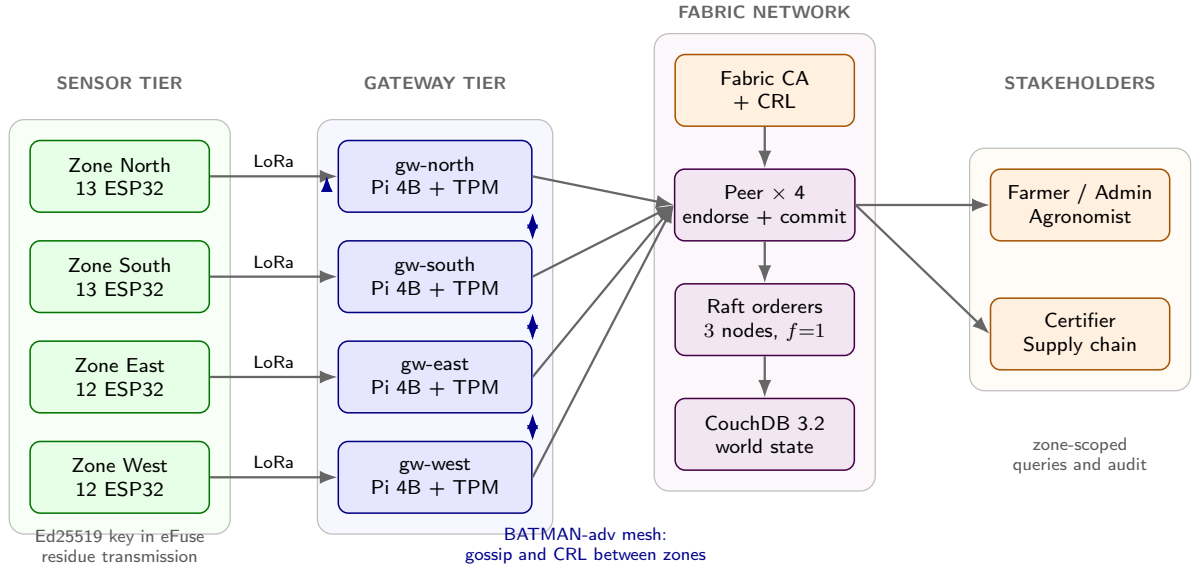


Figure 1: Deployment architecture. Sensors reach the ledger only through their zone gateway, which is also their registration authority during enrollment. Zone membership is carried in the device certificate and re-checked by chaincode on every transaction.

IoT and blockchain for climate-smart cultivation and, like most pre-harvest work, treat authorization as an implementation detail rather than a measured component. Liu et al. [23] and Wang et al. [45] reduce processing latency at the agricultural edge without addressing the authorization layer above it. Mohanta et al. [24] apply smart-contract enforcement in smart grids, a neighbouring domain with comparable device constraints and a similarly centralised starting point.

On identity infrastructure, W3C Decentralized Identifiers [46] and gateway-assisted verification for constrained devices [9, 40] offer an eventual migration away from X.509, which we treat as future work rather than a deployed alternative. On the radio side, Adelantado et al. [2] characterise the LoRaWAN capacity limits that bound achievable sensor density per gateway, a constraint that becomes visible in Section 6.7.

3. Access-control design

3.1. Deployment structure and threat model

The deployment has three tiers, shown in Figure 1. ESP32-WROOM-32 sensor nodes (dual-core Xtensa LX6 at 240 MHz, 520 kB SRAM) sample soil and environmental data, sign each reading with an Ed25519 key held in one-time-programmable eFuse, and transmit over LoRa. Raspberry Pi 4B zone gateways (quad-core Cortex-A72, 4 GB LPDDR4, TPM 2.0) receive those payloads, verify signatures, endorse Fabric transactions and cache policy decisions locally. A Fabric network of four peers, one per zone, three Raft orderers and CouchDB 3.2 for world state maintains the ledger. Gateways interconnect over a BATMAN-adv mesh [25] that reroutes around individual gateway failures and carries Fabric gossip and certificate revocation list (CRL) traffic between zones.

Following the permissioned-blockchain model in which every participant is credentialed [4], orderers and the certificate authority are operated by vetted institutional entities and treated as non-malicious. Raft gives crash-fault tolerance with $2f + 1$ nodes [28], so the three-node cluster survives one orderer failure but not a reachable orderer behaving maliciously. Byzantine-tolerant ordering [50, 7, 16] would widen that boundary at higher consensus cost.

Sensors, client devices and every wireless channel are treated as potentially compromised. Physical tampering and side-channel extraction against cryptographic accelerators sit outside the evaluated scope and are revisited in Section 8.4.

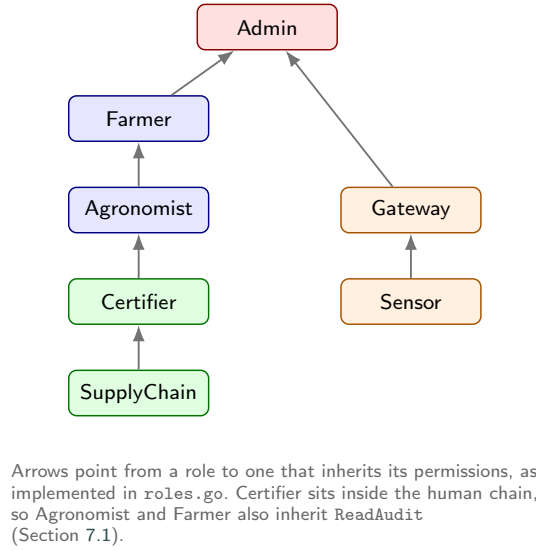


Table 2

Effective permissions per role, generated from the deployed chaincode (`roles.go`) rather than transcribed. Inheritance along the human chain of Equation (1) is why *Read audit* reaches Agronomist and Farmer.

Permission	Sensor	Gateway	Farmer	Agronomist	Certifier	Supply	Admin
Read own	–	–	✓	✓	✓	✓	✓
Read zone	–	✓	✓	✓	✓	–	✓
Read all	–	–	–	–	–	–	✓
Write sensor	✓	✓	✓	–	–	–	✓
Control zone	–	✓	✓	✓	–	–	✓
Control all	–	–	–	–	–	–	✓
Read audit	–	–	✓	✓	✓	–	✓
Issue CZT	–	–	✓	✓	–	–	✓
Manage roles	–	–	–	–	–	–	✓
Admin all	–	–	–	–	–	–	✓

where $pa(r)$ are the permissions assigned directly to r and $active(u)$ the non-expired roles held by u . An access request (u, op, res, t) is granted iff

$$\begin{aligned} &\exists r \in active(u, t) : (op, res) \in eff(r) \\ &\wedge (zone(u) = zone(res) \vee czt(u, zone(res), t)), \end{aligned} \quad (4)$$

where $czt(u, z, t)$ holds iff u has a valid administrator-issued cross-zone token for z at time t , and $active(u, t) = \{r \mid \exists assign(u, r, t_s, t_e) : t_s \leq t \leq t_e\}$. Table 2 gives the permission matrix that results.

3.3. Zone and temporal scoping

Geographic farm regions (North, South, East, West) define the zone model, and it is enforced at three independent points rather than one. Each device certificate carries its zone in an X.509 Subject Alternative Name extension under a private OID arc. Each gateway accepts certificate signing requests only from sensors declaring its own zone. At the chaincode level every zone-scoped transaction calls `validateZoneAccess`, which parses the caller's zone from the certificate DER bytes by ASN.1 decoding rather than by matching against the PEM text. That last detail matters: string matching against a common name is spoofable by an attacker who controls the subject field, and ASN.1 decoding of the extension is not.

Cross-zone tokens are administrator-issued and recorded immutably with subject, permitted zones, validity window and purpose, so a temporary cross-zone grant expires on its own rather than requiring a reissued certificate.

Temporal constraints run in two independent layers. The chaincode compares an `ExpiresAt` field on the role assignment against the Fabric transaction timestamp before every decision, and the Fabric MSP independently validates the X.509 `NotAfter` field on every incoming transaction. A seasonal worker granted the Farmer role for a harvest window therefore loses access at expiry without an administrator doing anything, and the expiry is recorded as a new audit entry referencing the original assignment. Gateway GPS modules supply a sub-microsecond time reference, distributed onward by NTP, comfortably inside the ± 5 -minute enforcement window.

Algorithm 1 states the decision procedure that Equation (4) describes. Two properties of it matter for the measurements in Section 6.1.

The work is bounded and structurally independent of which operation was requested: the hierarchy is walked to a fixed depth of at most three, and the zone check is a single comparison against a parsed certificate field. That is a property of the code, and it is the mechanism we would expect to produce similar costs across operations.

Every path through the algorithm writes an audit entry, including every denial, so a `CheckAccess` invocation is state-mutating and commits through ordering exactly as a sensor write does. It is easy to assume a permission check is a read; in this design it is not. Section 6.1 therefore compares two classes of committed transaction, neither of which avoids consensus. Expired assignments are revoked inside the same

Algorithm 1 Access decision (**CheckAccess**)**Input:** request (u, op, res, t) , caller certificate C **Output:** GRANT or DENY. Every path writes an audit entry, so every invocation mutates state and commits through ordering.

```

1:  $n \leftarrow \text{NONCE}(req)$ 
2: if SEENBEFORE( $n$ ) then
3:   AUDIT(DENY, REPLAYDETECTED); return DENY
4: end if
5:  $z_u \leftarrow \text{PARSEZONEASN1}(C)$  ▷ DER bytes, not PEM text
6: if ONCRL( $C$ ) then
7:   AUDIT(DENY, CERTREVOKED); return DENY
8: end if
9:  $A \leftarrow \text{ROLEASSIGNMENTS}(u)$ 
10:  $R_{\text{act}} \leftarrow \emptyset$ 
11: for  $a \in A$  do
12:   if  $a.\text{ExpiresAt} < t$  then
13:     REVOKEROLE( $a$ ) ▷ state-mutating: forces endorsement
14:   else
15:      $R_{\text{act}} \leftarrow R_{\text{act}} \cup \{a.\text{role}\}$ 
16:   end if
17: end for
18: if  $R_{\text{act}} = \emptyset$  then
19:   AUDIT(DENY, ROLEINSUFFICIENT); return DENY
20: end if
21:  $P \leftarrow \bigcup_{r \in R_{\text{act}}} \bigcup_{r' \preceq r} pa(r')$  ▷ inheritance closure, depth  $\leq 3$ 
22: if  $(op, res) \notin P$  then
23:   AUDIT(DENY, OPNOTPERMITTED); return DENY
24: end if
25:  $z_r \leftarrow \text{ZONEOF}(res)$ 
26: if  $z_u \neq z_r \wedge \neg \text{VALIDCZT}(u, z_r, t)$  then
27:   AUDIT(DENY, ZONEMISMATCH); return DENY
28: end if
29: RECORDNONCE( $n$ ); AUDIT(GRANT,  $(op, res, z_r)$ )
30: return GRANT

```

transaction that denies the request, which keeps the audit trail single-sourced at the cost of making even a read-intent check a write.

3.4. Cascading enrollment

An ESP32 cannot hold the TLS handshake state that direct interaction with a certificate authority requires: the mbedTLS handshake path on ESP-IDF consumes roughly 70 kB of heap against a 520 kB SRAM budget shared with the application, consistent with the constrained-device measurements of Raza et al. [32]. The protocol therefore pushes registration authority down the tier structure in three stages (Figure 3): long-lived administrator credentials, then gateway registration with the private key in TPM 2.0, then sensor enrollment with the gateway acting as local registration authority (Supplement S1).

Because the private key is generated on the sensor and never transmitted, compromising a gateway does not yield sensor signing keys. Re-provisioning a node whose key is lost needs physical access, since eFuse programming is irreversible.

The key is *not* read-protected in the deployed firmware. Signing uses wolfSSL in software, so `private_key_read` copies the 32 bytes out of eFuse block 3 into RAM for each signature and zeroes the buffer afterwards. Setting the eFuse read-disable bit would make that copy fail and break signing outright, because the ESP32 used here has no hardware Ed25519 engine and no key-wrapping peripheral that could sign on the key's behalf. Read protection is a hardening step that a different part is needed to support, for instance the HMAC and digital-signature peripherals on the ESP32-S3, rather than something the deployed nodes achieve. The eFuse gives durability and resistance to reflashing, not extraction resistance against code

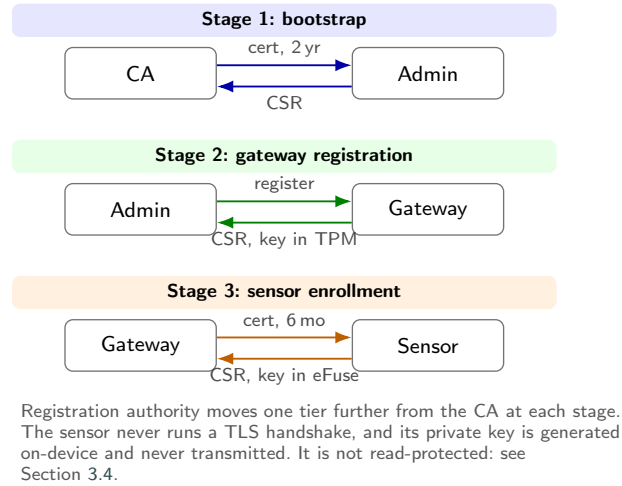


Figure 3: Three-stage cascading enrollment.

running on the device, and Section 8.4 counts key extraction by an attacker with code execution as residual risk.

4. Implementation

4.1. Chaincode and transaction flow

The chaincode is written in Go 1.21 against `fabric-contract-api-go` and targets Fabric 2.5. Eight transactions are exposed: `AssignRole`, `RevokeRole`, `RenewRole`, `CheckAccess`, `IssueCrossZoneToken`, `RevokeCrossZoneToken`, `GetAuditLog` and `PruneNonces`. Figure 4 places the access-control checks inside Fabric's execute-order-validate pipeline.

Three implementation decisions shape the measurements. Every state-mutating call carries a UUID nonce recorded in world state, rejected on reuse before any state is read, which defeats replay inside the timestamp window. Every access attempt, granted or denied, writes an audit entry holding the caller's certificate hash, operation, resource, zone, decision and reason, so the audit trail is a byproduct of enforcement rather than a parallel log that can drift from it. And `CheckAccess` revokes an expired assignment in the same transaction that denies the request, which makes even a read-intent check state-mutating and therefore endorsed rather than served as a query.

Peer configuration for the ARM64 gateways uses 50 transactions per block, a 500 ms batch timeout, a 256 MB state cache, endorsement restricted to zone-local peers, Snappy compression on LevelDB and 5-minute gRPC keepalive. These target the write amplification that dominates latency on microSD-backed storage, and they sit within the parameter space Abang et al. [1] identify as governing Fabric latency.

4.2. Sensor firmware and residue-coded transmission

The sensor certificate lives in battery-backed RTC slow memory so it survives deep sleep, guarded by a CRC and a magic sentinel checked on each wake. A mismatch forces re-enrollment rather than transmission under an unverifiable identity. Signing uses wolfSSL's Ed25519 implementation. The firmware first hashes the payload with SHA-256, using the ESP32 hardware peripheral, and then signs the resulting 32-byte digest. This is a prehash construction and not standard Ed25519, which hashes the message internally with SHA-512, nor Ed25519ph, which prehashes with SHA-512 and uses a distinct domain separator. We flag the deviation because it matters for interoperability: a verifier must apply the same SHA-256 prehash, and the scheme inherits SHA-256's collision resistance rather than SHA-512's. Nothing in the deployment depends on the difference, but a reimplemention from the paper alone would not interoperate unless the prehash is reproduced. The main loop wakes every 30 minutes, verifies the cached certificate, samples the sensors, signs the payload, transmits and returns to deep sleep.

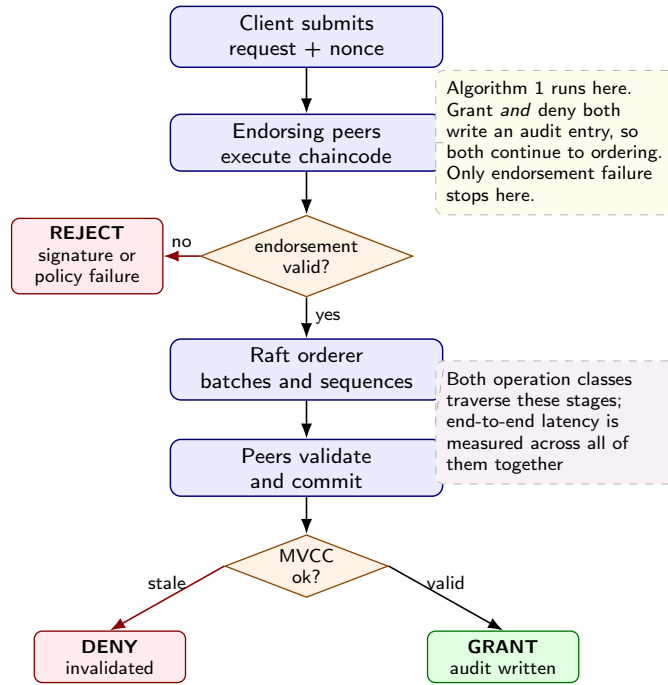


Figure 4: Access control inside the execute-order-validate flow. An authorization denial is a successful chaincode response carrying **DENY** plus an audit write, so it is ordered and committed like any other transaction. Only endorsement-level failures stop before the orderer.

Readings are transmitted as residues rather than as plaintext values. A value is reduced modulo the pairwise-coprime set $\{97, 101, 103\}$ and the three residues go out as separate single-byte packets, any two of which reconstruct the original by the Chinese Remainder Theorem [12] (Supplement S2).

The admissible range is where this scheme goes wrong, and the deployed firmware got it wrong; recovery from an arbitrary *pair* is unique only below the smallest pairwise product of the moduli, not below their product. Section 7.2 sets out the consequence. The corrected firmware uses $\{253, 254, 255\}$, giving a pairwise bound of 64 262, and quantises soil moisture from twelve bits to eight so the packed value reaches 51 455 and stays inside it. Keeping twelve bits would require all three residues and forfeit the redundancy entirely.

4.3. Gateway services and revocation propagation

Each gateway runs Fabric 2.5 from official ARM64 binaries. A Python enrollment service bound to the local interface accepts CSRs only from sensors in its own zone and rate-limits per source address. A policy cache with a 301-second nominal TTL absorbs repeated checks from the same sensor during a transmission burst.

Revocation is the mechanism whose behaviour we most wanted to observe over a long window, and Supplement S3 gives the path it takes.

The branch at line 7 is the whole story of Section 6.5. For a reachable gateway the cache is invalidated by the CRL event itself, so revocation staleness is bounded by gossip propagation and the nominal TTL never binds. For an unreachable one the bound falls back to the scheduled refresh interval. Which branch a given revocation takes is decided by connectivity, not by anything in the protocol.

4.4. Formal specification

A three-role subset of the model is specified in TLA+ and checked with TLC [20, 26], which finds no violation of the role-permission invariant and confirms that pending requests resolve under weak fairness. We describe this as a sanity check on a small model rather than verification of the deployed framework, for three reasons: the instance covers three of seven roles, zone and temporal predicates are absent, and nothing connects the specification to the Go implementation. Given Section 7.1, that last point is the binding one,

Table 3

Deployment configuration and observed totals.

Parameter	Value
Site	2 ha farm, Sfax, Tunisia (34.74°N, 10.76°E)
Duration	61 days, 1 Aug to 30 Sep 2025
Zones	4 (North, South, East, West)
Sensor nodes	50 ESP32-WROOM-32
Gateways	4 Raspberry Pi 4B (4 GB, TPM 2.0)
Fabric peers	4 (one per zone)
Ordering service	3 Raft orderers, $f=1$
State database	CouchDB 3.2
Sensor duty cycle	30 min
Peak ambient temperature	39.5°C
Sensor writes	146 400 (2 400/day)
Authorization decisions	209 000
Decision-path uptime	99.36%
Connectivity outages	3 (0.2–8.7 h, 9.3 h total)

since the implemented hierarchy differs from the specified one. The specification and its configuration are in the supplement and the artifact.

5. Measurement methodology

5.1. Site and workload

The system ran continuously for 61 days on a 2 ha experimental farm in Sfax, Tunisia (34.74°N, 10.76°E), from 1 August to 30 September 2025. Table 3 gives the configuration. Sfax is a useful site for this kind of measurement precisely because it is unforgiving: ambient temperatures reached 39.5°C during the trial, and rural connectivity is intermittent enough that the outage behaviour in Section 6.8 reflects ordinary operation rather than injected faults.

Each sensor reported on a 30-minute duty cycle, giving 2 400 sensor writes per day and 146 400 over the deployment. Human and administrative traffic, meaning farmer and agronomist queries, certifier audits, supply-chain provenance reads and administrative role operations, brings the total to 209 000 recorded authorization decisions, of which 4 860 (2.33%) were denials arising in ordinary operation rather than in the attack campaign.

5.2. Peer-scaling topology

Section 6.4 varies *configured logical peer count*, not physical capacity, and Table 4 is deliberately specific about what that means, since increasing configured peers does not automatically reduce Fabric latency: the effect depends on the endorsement policy, how the client selects endorsers, connection pooling, proposal fan-out, and peer CPU and state-database load. Only the 4-peer row is the physically deployed network in Table 3, one peer per zone on its own Raspberry Pi. The 8-, 16- and 32-peer rows run additional logical peers on the same four physical hosts, oversubscribed rather than added; the comparison therefore isolates the effect of endorser multiplicity under a fixed endorsement policy, not of adding physical machines.

5.3. Instrumentation

Nine trace streams were recorded. Per-decision records capture caller identity, role, operation, resource, zone, decision, denial reason and end-to-end latency. Per-transaction sensor records add signature validity, the number of residues received and measured transmission energy. Dedicated campaigns cover write-path latency against endorsing-peer count, throughput against offered concurrency for both the HRBAC and the access-control-free configuration, LoRa energy in single-channel and residue modes, microsecond-level cryptographic timings, revocation lifecycle events, authorization-boundary attack attempts and outage events.

Table 4

Topology and routing for each peer-scaling configuration. The endorsement policy, orderer topology, batch parameters and state database are identical across rows; only configured logical peer count and its physical placement change.

Parameter	Value
Configured logical peers	4, 8, 16, 32
Physical hosts	4 (Raspberry Pi 4B, one per zone)
Peers per host	1, 2, 4, 8 respectively
Endorsement policy	OR('FarmMSP.member'), one endorsement
Endorsements required	1 per transaction, any peer
Proposal selection	Fabric Gateway service default (discovery-based); not application-controlled
Load balancing	Delegated to Gateway service discovery; the client issues one submission per transaction
State database	CouchDB 3.2, one instance per peer
Block cut	BatchTimeout 2s, max 10 messages, 512 kB preferred size
Orderer topology	3 Raft orderers, $f=1$, unchanged across rows
CPU / memory allocation	No cgroup limits set; peers above 4 share the host of the corresponding zone's deployed peer

Table 5

Experimental unit used for each analysis in this article.

Analysis	Independent unit
Throughput vs. concurrency (Section 6.3)	Independent benchmark run (5 per cell)
Access-control invariance across peer counts (Section 6.1.2)	Single continuous run per configuration; block-bootstrapped
Peer-count latency comparison (Section 6.4)	Single continuous run per configuration; block-bootstrapped
Operation-label diagnostic (Section 6.2.1)	Deployment day (61 days)
Revocation stage records (Section 6.5)	Reported descriptively; stages do not resolve into independent episodes
Security campaign (Section 6.6)	Executed test corpus, not a random sample of attacks; no interval given

Latency is measured at the gateway as the interval between submission and commit notification, so it covers endorsement, ordering, validation and commitment, and excludes only the LoRa hop. Throughput is committed transactions divided by the measurement window. Energy is measured with an INA219 current sensor sampling at 2 kHz. The baseline for overhead attribution in Section 6.3 is the identical Fabric network, workload and hardware with the access-control chaincode removed; that removal also removes the audit write, the nonce check and their state access, so the difference it produces estimates the cost of the complete access-control-enabled transaction path, not an isolated Boolean permission decision (Section 8.4).

5.4. Statistical treatment

Table 5 states, for every comparison in this article, what the independent experimental unit is. Getting this wrong is the most common way a measurement study overstates precision: transactions generated in the same run, day or gateway are correlated, and treating each as an independent replicate produces standard errors far smaller than the design supports.

Group comparisons where the groups are independent replicates use Welch's t -test, reported with the mean difference, its 95% confidence interval and Cohen's d . At these sample sizes almost any difference reaches significance, so the effect magnitude carries the information and the p -value mostly confirms that the sample is large. Comparisons across more than two independent groups use one-way ANOVA.

Two comparisons do not have independent replicates: each configuration in the peer-scaling campaign (Sections 6.1.2 and 6.4) is a single continuous benchmark run of consecutive, two-minutes-apart transactions rather than a set of repeated trials, so a Welch test or an exact order-statistic interval would treat serially correlated samples as independent and understate uncertainty. Both use a moving-block bootstrap instead:

contiguous blocks of length $\approx n^{1/3}$ (reported alongside every such interval, e.g. Table 9) are resampled rather than single points, which preserves whatever short-range correlation the run has. This describes the one run that was executed; with a single run per configuration there is no independent replicate to generalise from, and Section 8.4 returns to what that limits.

Confidence intervals on means from independent replicates use the Student- t interval where $n \geq 1000$ and a percentile bootstrap of 10 000 resamples below that. The ten throughput comparisons are corrected for multiplicity with the Holm-Bonferroni procedure; 10 of 10 remain significant after adjustment, and both the raw and adjusted p -values are reported (Table 8). The five runs per concurrency cell were not interleaved between the HRBAC and baseline arms: for a given concurrency, one arm's five runs all preceded the other's, so trial order and arm are not independent of each other. Pooling each run's deviation from its own cell mean against its position in that cell's run sequence detects no trend ($R^2 = 0.013$, $p = 0.27$), which is evidence against a warming or ledger-growth confound but does not rule one out by design.

We deliberately attach no interval to the block rate in Section 6.6. Repetitions of a scripted scenario are not independent samples from a population of attacks, and any interval over them would characterise the repetition count rather than the security property. The same reasoning applies to the revocation stage records, which are correlated within an episode and, as Section 7.4 explains, do not resolve into episodes at all.

Equivalence claims use two one-sided tests (or, for the two single-run comparisons above, the block-bootstrap interval against the same margin) rather than the absence of a significant difference, since a non-significant test establishes only that nothing was detected. The ± 10 ms margin was fixed before this reanalysis of the released trace, after the data were already available to us; it is not a preregistered value recorded before the trace was collected, and we report it that way rather than as a prespecified design parameter. Operationally, 10 ms is about 3.5% of the 280 ms decision cost, well below anything that would change a provisioning decision about where to spend engineering effort.

5.5. Reproducibility

Every number, table and figure in this article is generated from the released traces by one script. Running

```
python3 analysis/derive_results.py
```

reads the raw traces, recomputes the statistical summary and emits both a machine-readable results file and the LaTeX macros and table bodies this manuscript expands. Nothing measured was transcribed by hand, so the text cannot drift away from the dataset. A test in the repository asserts that every macro in the manuscript matches the corresponding derived value, so the guarantee is checked rather than asserted.

Because a repository URL alone does not pin what a result was computed from, the pipeline emits a SHA-256 digest, byte count and row count for each of the 10 input traces (375 854 rows in total), recorded in the released results file. Resampling uses a fixed seed. The submission corresponds to a tagged release, `v07-submission`, identified by its Git commit SHA and archived through Zenodo's GitHub integration, which mints a version DOI at release time; readers should cite the tag, commit and DOI recorded on the archived release page rather than the repository's default branch, which continues to move. The release carries a pinned environment (Python and package versions in `analysis/requirements.txt`, TeX package versions in the build log) so that `bash paper_iot/build_cas.sh` regenerates every macro, table and figure and recompiles the manuscript in one command, and a data dictionary (`data/raw/README.md`) marks the fields withdrawn or reinterpreted in this revision: signature validity, the CRT recovery count, and the topology trace's coordinate columns. A second driver, `analysis/run_campaign.py`, repeats the throughput, latency and decision-cost campaigns against a live Fabric network and writes its output in the same schema, so measurements from an independent installation can be substituted and the article regenerated against them. The energy and cryptographic-timing traces are not reproduced by that driver, since they need the INA219 instrumentation and ESP32 bench described above, and it does not synthesise numbers it cannot measure.

6. Results

6.1. Latency accounting for the write path

Both transaction classes in the trace are endorsed, ordered and committed, since as Section 4.1 explains every authorization outcome writes an audit entry. The interesting question is not which class is slower but how much of a write is associated with the access-control layer, and the chaincode logs a timing field on every write towards answering it: `rbac_overhead_ms`, recorded alongside end-to-end latency.

6.1.1. A descriptive latency account, not a stage decomposition

Write the write-path latency at configured logical peer count p as

$$L(p) = A + Q(p) + F, \quad (5)$$

where A is the recorded authorization-associated span, $Q(p)$ is whatever part of the remainder responds to peer provisioning, and F is whatever does not. This is a bookkeeping identity, not a claim that A , Q and F are independently identified Fabric stages: Q and F are defined as residuals of A , and Section 6.1.3 is explicit about what that does and does not establish.

A is the recorded span. Averaged over the campaign it is 319ms. We deliberately do not call this a direct measurement of permission-evaluation cost. The deployment recorded a historical chaincode timing field labelled `rbac_overhead_ms`; because its exact instrumentation boundaries, the instruction immediately before the timer starts and immediately after it stops, cannot be reconstructed from the deployed source (Section 8.3), we treat it as an authorization-associated implementation span rather than an identified permission-evaluation cost.

One comparison needs explaining before going further, because it looks contradictory. The recorded span inside a write, 319ms, is larger than a complete standalone `CheckAccess` transaction, 280ms (Section 6.2). The two counters do not share a documented boundary: `rbac_overhead_ms` is recorded inside the endorsing peer around the policy evaluation, including the world-state reads it triggers, whereas the standalone figure is an end-to-end client-observed latency for a transaction that carries a smaller payload and different state access. We cannot rule out that the two also differ in what they include for reasons beyond payload and state access, since neither counter's boundaries are documented at the instruction level. Section 8.3 lists what a reimplementation would need to make the two comparable.

Q follows from using peer count as a lever rather than from fitting a model. Writing $R(p) = L(p) - A$ for the remainder, $Q = R(4) - R(32) = 397\text{ms}$ is the part of the remainder that an eightfold increase in configured peers removes. No functional form is assumed and nothing is extrapolated, and Q is not claimed to be endorser queueing specifically: Section 6.4 returns to that as a hypothesis rather than a measured mechanism.

$F = R(32) = 503\text{ms}$ is the remainder at the largest configuration measured. We are careful with what this is. It is a baseline, not an identified peer-invariant floor and not a measured platform-configuration cost: the remainder was still falling at 32 peers, and its decrements per doubling were growing rather than shrinking (73.5, 94.5 and 228.7ms), so nothing in these measurements indicates a plateau. Establishing one would need configurations beyond 32 peers on genuinely distinct physical hosts (Table 4). Read F as an upper bound on any floor, and Q as the reduction that scaling to 32 peers achieved rather than the total reduction available.

The three terms sum to the observed 1220ms by construction, since Q and F are both defined from A ; substituting the definitions leaves $A_{\text{pooled}} - A(4)$, so the 0.9ms residual restates the invariance of A rather than confirming the accounting independently. That closure is arithmetic, not evidence. The empirical content is in the invariance of A , tested next. Table 6 gives the descriptive account per peer count: total latency, the recorded span, what remains once that span is subtracted, and how much that remainder falls relative to the smallest configuration. Figure 5 plots the same three quantities.

6.1.2. The recorded span is stable in configured peer count

That A can be treated as approximately constant is a measured property of this run, not an assumption of the accounting. Across the four configured peer counts the per-configuration means span 3.4ms, from 317.7ms at 32 peers to 321.1ms at 8 peers, without a monotonic trend between them. Regressing on $\log_2 p$

Table 6

Write-path latency accounting at 50 concurrent clients. *Recorded authorization span* is `rbac_overhead_ms`, instrumented on every write; *remaining latency* is total latency minus that span, with no component inside it separately measured. *Reduction vs. 4 peers* is the decrease in the remaining latency relative to the smallest configuration. Values in ms; see Section 6.1.

Configured peers	Total latency	Recorded authorization span	Remaining latency	Reduction in remainder vs. 4 peers
4	1219.8	320.3	899.5	0.0
8	1147.1	321.1	826.0	73.5
16	1050.5	319.0	731.5	168.0
32	820.5	317.7	502.8	396.7

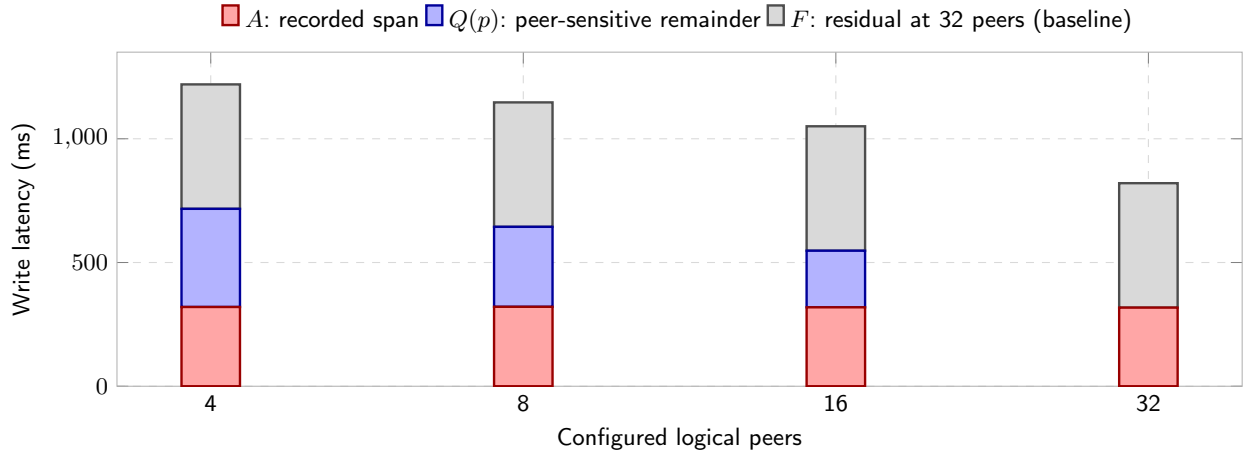


Figure 5: The latency account of Equation (5) across configured peer counts. The recorded-span band is nearly the same height throughout; the peer-sensitive band shrinks to zero at 32 peers by construction, since F is defined as the remainder there, not because the underlying cost is known to vanish.

gives 0.98 ms per doubling with $p = 0.14$, so no trend is detected; since a null result establishes nothing on its own, the extreme configurations (4 and 32 peers) are compared with a moving-block bootstrap rather than an independent-samples test, because each configuration is one continuous run rather than repeated independent trials (Section 5.4). Blocks of 9 and 9 consecutive samples respectively give a mean difference of 2.58 ms with a 90% interval of $[-4.51, -0.43]$ ms, entirely inside the ± 10 ms margin, matching the two-one-sided-tests convention used elsewhere in this article. This describes the two runs that were executed; a single run per configuration cannot establish that repeating the same configuration would reproduce the same mean, and configuration order was not randomized (Section 8.4).

The engineering consequence, read with that caveat, is direct. The recorded span is 26.2% of the write path at 4 peers, and because it does not shrink when peers are added, its share *rises* to 38.7% of the smaller total at 32 peers. Optimising it is bounded above by roughly a quarter of write latency in the deployed configuration, and by rather more once peers have been scaled out, which is the opposite of what a designer would assume from the 4-peer figure alone.

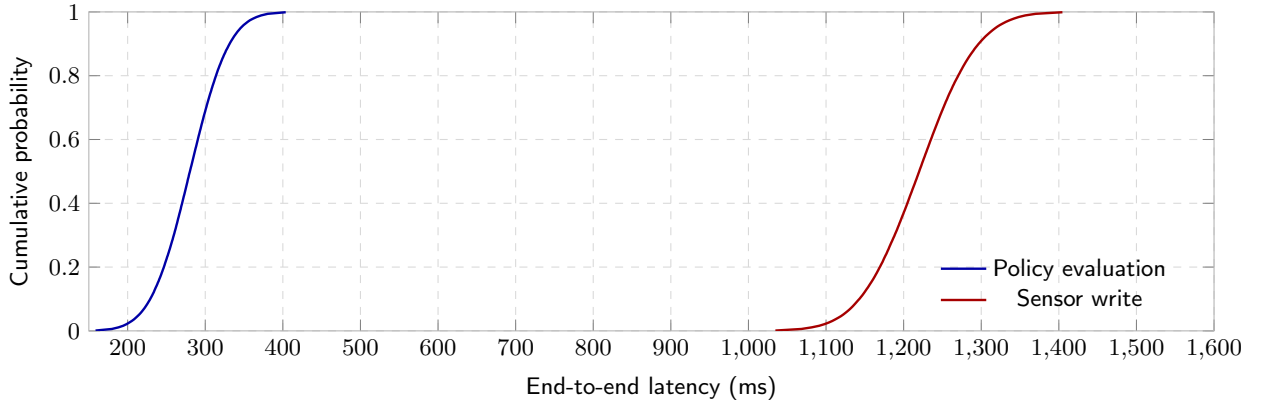
6.1.3. What the partition does not separate

F aggregates ordering, validation, commit and whatever part of endorsement execution does not respond to peer count. Splitting it needs a transaction timestamped at each Fabric hand-off, which we did not collect, and no amount of reanalysis of these traces will produce it. The same applies to naming which resource sets the throughput ceiling in Section 6.3: the accounting locates cost, it does not identify components inside F .

Table 7

End-to-end latency by operation class (ms). Both rows are committed transactions: an authorization decision, permitted or denied, always writes an audit entry (Section 4.1). The 7 `CheckAccess` operation types are pooled here rather than broken out, because the recorded labels do not separate policy-distinct executions (Section 6.2.1).

Operation	n	Mean	SD	P95	P99
<code>CheckAccess</code> (pooled, committed)	62 600	280.3	39.9	345.8	—
<code>WriteSensor</code> (committed)	146 400	1219.8	60.0	1318.6	1358.9

**Figure 6:** Empirical CDFs of the two committed transaction classes. Both traverse ordering and commitment.

Read with that boundary in mind, Equation (5) answers the question a designer actually faces, which is how much of the budget is associated with the access-control layer, without claiming a stage attribution the measurements cannot support.

6.2. Latency by operation class

Table 7 gives end-to-end latency for the two committed transaction classes. Policy evaluations (`CheckAccess`, pooled across the 7 operation types recorded) average 280 ms (SD 40) and sensor writes 1 220 ms (SD 60, P95 1 319, P99 1 359), a difference of 940 ms. Figure 6 shows the two distributions separating cleanly, the write distribution both shifted and wider.

The 940 ms gap is a difference between operation classes that differ simultaneously in payload size, world-state access and endorsement work. Consensus is common to both, so it is not the cost of consensus, and it should not be confused with the latency accounting above, which is derived within the write path rather than across classes.

6.2.1. Why an operation-level breakdown is not reported

The 7 `CheckAccess` operation types recorded in the operational trace (`ReadOwn` through `ManageRoles`) are pooled in Table 7 rather than broken out, and it is worth saying why rather than leaving a reader to wonder whether we simply did not look. Across the 35 role-by-operation cells in the operational decision trace, the grant rate is uniformly between 91% and 93%. A trace in which the recorded operation and role determined the policy path would show cells near 0% or near 100%, matching the permission matrix in Table 2. Uniform rates mean the operation and role labels in this stream do not separate policy-distinct executions, so a comparison between them, however it is analysed, groups one largely homogeneous population by an uninformative label rather than testing whether privilege level costs anything.

The defensible claim is therefore narrow: committed `CheckAccess` transactions averaged 280 ms (SD 40) across the trace; the recorded labels do not support an operation-specific or privilege-specific comparison. Whether privilege level or hierarchy depth affects decision cost needs an experiment that varies them

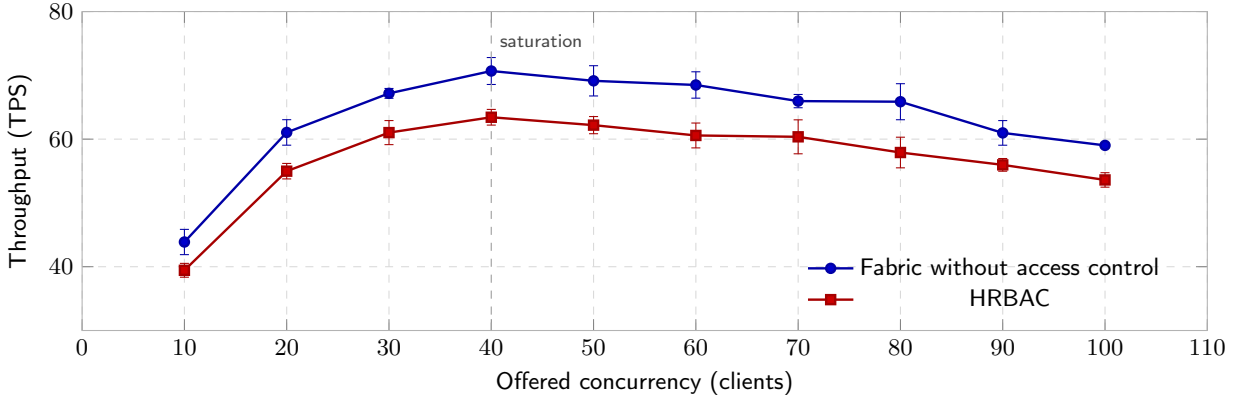


Figure 7: Throughput against offered concurrency for both arms, with standard deviations over five runs per point. Both curves peak at the same offered load, which places the bottleneck below the access-control layer.

Table 8

Throughput by offered concurrency (TPS, five independent runs per cell, mean $\pm$ SD). Reduction is the HRBAC cost relative to the baseline; p from Welch's t -test, unadjusted and Holm-adjusted across the ten concurrency levels. Runs were not interleaved between arms (Section 6.3).

Clients	Baseline	HRBAC	Reduction	p	p_{Holm}
10	43.9 $\pm$ 2.0	39.4 $\pm$ 1.1	10.1%	0.004	0.009
20	61.0 $\pm$ 2.0	55.0 $\pm$ 1.2	9.9%	<0.001	0.006
30	67.2 $\pm$ 0.8	61.0 $\pm$ 1.9	9.2%	<0.001	0.006
40	70.7 $\pm$ 2.1	63.4 $\pm$ 1.2	10.3%	<0.001	0.003
50	69.1 $\pm$ 2.4	62.2 $\pm$ 1.3	10.0%	0.001	0.006
60	68.5 $\pm$ 2.1	60.6 $\pm$ 1.9	11.6%	<0.001	0.003
70	66.0 $\pm$ 1.0	60.4 $\pm$ 2.7	8.5%	0.007	0.009
80	65.9 $\pm$ 2.8	57.9 $\pm$ 2.4	12.1%	0.001	0.006
90	61.0 $\pm$ 1.9	56.0 $\pm$ 1.0	8.2%	0.002	0.006
100	59.0 $\pm$ 0.2	53.6 $\pm$ 1.1	9.2%	<0.001	0.003

deliberately with the workload otherwise fixed, which Section 8.3 sets out. The scripted campaign in Section 6.6, where every boundary-crossing request was refused, is the evidence that the policy itself discriminates.

6.3. Throughput and where the ceiling sits

Figure 7 and Table 8 report throughput against offered concurrency for the HRBAC configuration and the access-control-free baseline on identical hardware and workload.

Two features of these curves matter more than the peak values themselves.

The access-control cost is a constant factor rather than a load-dependent one. Across the ten concurrency levels the reduction averages 9.9% with a standard deviation of 1.2 percentage points, ranging from 8.2% to 12.1%. Regressing reduction on offered concurrency gives $R^2 = 0.020$ with $p = 0.69$, so no trend is detectable. Pooled across all runs the difference is significant ($t = 4.3$, $d = 0.86$) and it does not grow with load. For capacity planning this is the well-behaved case: budget roughly a tenth of platform throughput for authorization and expect that fraction to hold as load rises.

Both arms then saturate at the same offered load. Peak throughput is 70.7 TPS for the baseline and 63.4 TPS for HRBAC, both at 40 clients, after which both decline, HRBAC by 15.5% out to 100 clients. Were authorization the limiting resource, adding it would move the knee to a lower offered load. It does not move at all, which places the limit below the access-control layer.

Table 9

Write-path latency against configured logical endorsing-peer count at 50 concurrent clients. Each row is one continuous benchmark run rather than repeated independent trials, so the P95 interval uses a moving-block bootstrap (block length in samples, rightmost column) rather than the exact order-statistic construction, which assumes independent observations.

Peers	n	Mean (ms)	P95 (ms)	P95 95% CI	Block ℓ
4	800	1219.8	1311.3	[1301.9, 1323.8]	9
8	500	1147.1	1237.2	[1231.1, 1245.3]	8
16	500	1050.5	1142.4	[1136.0, 1147.8]	8
32	800	820.5	903.2	[894.2, 911.3]	9

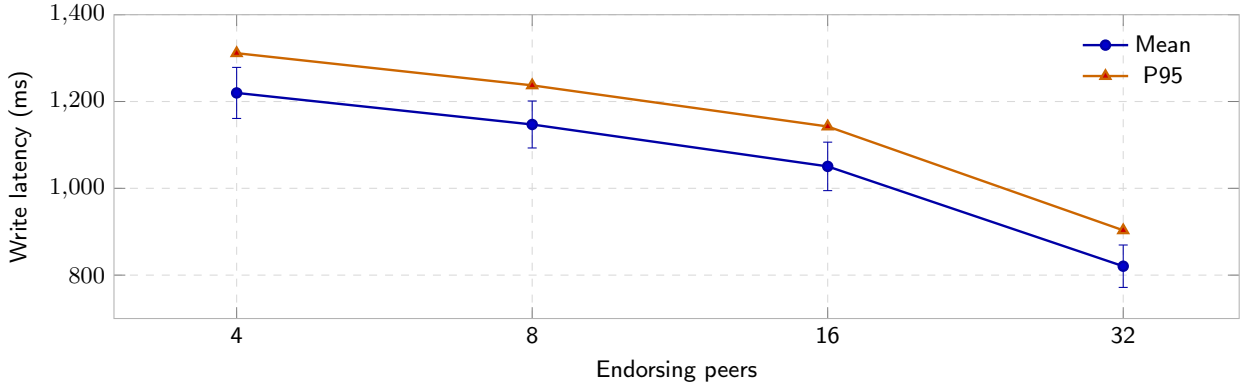


Figure 8: Write latency falls close to log-linearly with configured logical peer count (-129.4 ms per doubling, $R^2 = 0.925$). Peer-count throughput was not measured in this design (Section 6.4); the concurrency-throughput experiment in Section 6.3 used a fixed peer configuration.

We stop there rather than naming the resource. The accounting in Section 6.1 does not isolate ordering: F aggregates ordering, validation, commit and whatever part of endorsement execution does not respond to peer count. Ordering saturation is one candidate and Thakkar et al. [41] report it for Fabric on x86, but so are CouchDB service times, validation and commit capacity, endorsement queueing and gateway networking. Separating them needs orderer queue depth, block-fill rate, validation backlog and disk latency alongside a controlled intervention, none of which we recorded. These clients are also closed-loop, so offered concurrency is a client count rather than a controlled arrival rate. The decline past the knee is consistent with queueing beyond saturation and argues for admission control at the gateway rather than added capacity.

6.4. Peers trade latency, not throughput

Endorsing peers are normally added to raise capacity. Here they do something else. Table 9 and Figure 8 report write latency as the peer set grows from 4 to 32 at fixed offered concurrency.

Mean write latency falls from 1220 ms to 821 ms, a reduction of 32.7%. Each configuration is one continuous benchmark run rather than repeated independent trials (Table 5), so the difference is reported as a moving-block bootstrap mean difference of 399.2 ms, 95% interval [393.8, 404.5] ms (block lengths 9 and 9 samples), rather than as a Welch t -test, which would treat consecutive two-minute samples as independent replicates. P95 falls from 1311 ms to 903 ms (Table 9, block-bootstrap intervals). The decline is close to log-linear at -129.4 ms per doubling with $R^2 = 0.925$. This campaign did not measure throughput as a function of peer count, only latency, so we do not report whether throughput changed alongside it; the separate concurrency-throughput comparison in Section 6.3 used a fixed peer configuration and is not informative about peer-count effects.

Endorser queueing is the explanation we favour: at four peers each zone has one endorser and concurrent requests from that zone serialise behind it, while at 32 peers each zone has eight. We did not record queue

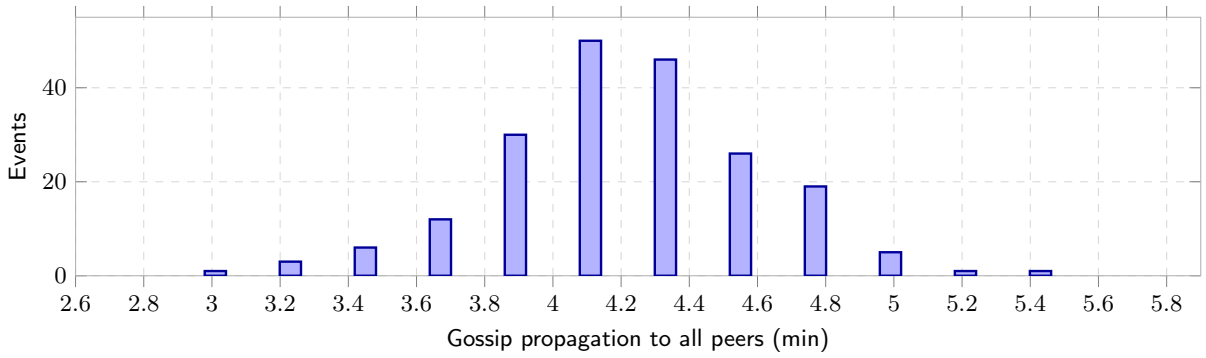


Figure 9: Gossip propagation across 200 revocation-lifecycle events.

Table 10

Revocation lifecycle by event type. Delay is a scheduling outcome, not an error: every event ultimately completed.

Event type	n	Delayed	Rate	Gossip (min)
CACHE_INVALIDATE	55	17	30.9%	4.24
CERT_REVOKE	53	13	24.5%	4.15
CRL_PUBLISH	46	18	39.1%	4.24
GOSSIP_COMPLETE	46	11	23.9%	4.22

depth or peer utilisation, so this is a hypothesis consistent with the shape of the curve rather than a measured mechanism, and the peer counts above four were hosted on shared physical hardware (Table 4), which varies logical endorser count rather than genuine physical distribution; configuration order is a further unexcluded confound (Section 8.4). For irrigation control with a two-second responsiveness budget [21], adding peers widens the margin against tail latency rather than lifting a capacity ceiling that sits downstream of the policy layer, if the queueing account is correct.

6.5. Revocation over a multi-week window

Revocation is where a short benchmark is least informative, because the mechanism fires rarely and its failure mode is delay rather than error. Over the deployment, 200 lifecycle *stage records* were logged across 58 days. These are stage observations rather than independent revocations, and Section 7.4 explains why end-to-end exposure cannot be recovered from them.

On the happy path the pipeline is quick. CRL publication completes in 60.4s on average (SD 4.8, maximum 71.6), and gossip to all peers takes 4.21 min (SD 0.38, P95 4.81, maximum 5.54). Figure 9 shows a tight unimodal propagation distribution with no heavy tail.

The longitudinal view is less comfortable. 59 of the 200 stage records, 29.5%, completed in a delayed state. We give the proportion without a confidence interval, since correlated stages of the same episode are not independent trials. Table 10 breaks this down by event type, and CRL publication is the weakest stage at 39.1% delayed against 23.9% for gossip completion.

Delayed stages are not slower on the metrics we can observe: their publication and gossip durations are statistically indistinguishable from successful ones. The conditioning matters, though. Our timers start when publication or gossip begins, so any interval spent waiting for a gateway to return is outside the measurement. The similarity says the propagation mechanism behaves the same once it starts, not that delayed revocations were operationally equivalent; the waiting time, which is what an operator cares about, was not recorded. Taken with the connectivity outages in Section 6.8, the pattern points to scheduling and reachability rather than to propagation degrading under load. If that reading is right, a faster gossip protocol would not help and event-driven status checking that survives disconnection would, but confirming it needs the end-to-end timing the trace lacks.

Table 11

Authorization-boundary campaign, grouped by the denial mechanism recorded rather than by scenario label, since the scenario names do not describe what was tested (Section 6.6). All 8 000 attempts were refused; no interval is given, since repeated scripted requests are not independent draws from an attack population.

Observed denial mechanism	Count	Roles	Zones	Identities
Policy violation	1416	6	4	467
Insufficient role	1353	6	4	464
Revoked certificate	1340	6	4	462
Operation not permitted	1313	6	4	463
Gateway rate limit	1304	6	4	456
Zone mismatch	1274	6	4	465
Total	8000			

Enforcement itself held throughout. In operational traffic, 973 access attempts were denied with revocation as the recorded cause, 16.0 per day across the 61-day deployment, spread out rather than concentrated in any single incident. The scripted campaign of Section 6.6 contributed a further 1340 revocation denials; we keep the two separate because one is field observation and the other is synthetic testing.

The residual exposure follows from Supplement S3 directly. For a connected gateway, staleness is bounded by gossip propagation, a few minutes. For a gateway disconnected when a revocation is issued, the bound is the scheduled CRL refresh interval, up to 24 hours. With 29.5% of events taking the second branch in practice, that upper bound is not hypothetical, and we treat it as the principal operational weakness of the present design rather than as an accepted trade-off.

6.6. Authorization boundary enforcement

The security evaluation targets the property the system exists to provide, that no principal acts outside its role, zone or validity window. 8 000 scripted attempts ran, each crossing one specific boundary of the model in Section 3, organised at collection time into 8 named scenarios of 1 000 attempts.

We do not report Table 11 by those scenario names, and it is worth saying why. Repetitions of a scripted request are not independent draws from a population of attacks, so no confidence interval is attached to the block rate. More seriously, the scenario labels do not describe the operations recorded against them: the `agronomist_irrigate` rows, for instance, contain `ControlZone`, `ReadAudit`, `WriteSensor`, `ReadAll` and `ManageRoles` requests spread across all four zones, so the label cannot be read as a description of what was tested. Table 11 instead groups every attempt by the denial mechanism the gateway or chaincode actually recorded, which is a fact about the trace rather than an label assigned at collection time.

All 8 000 attempts were refused, and that is close to everything this campaign establishes. What survives is narrow and worth stating exactly: 8 000 requests that crossed some boundary of the deployed policy were refused, none was granted, and the refusals invoked 6 distinct denial mechanisms rather than concentrating in one check, each spanning every role and zone in the deployment (Table 11). That breadth is the interesting part: a suite in which every attempt is caught by the same check would demonstrate one working mechanism, not a working model. The rate-limiting mechanism is recorded as `IDS_BLOCKED`, which is a misleading name for the enrollment and request rate limiter of Section 4.3; no anomaly-detection component exists in the design. So the campaign is evidence that the deployed policy denied what it was asked to deny. It is not a scenario-by-scenario conformance result, and combined with Section 7.1 it is not evidence about the intended design either.

A further limitation applies regardless of labelling. The expected-permission oracle against which each attempt is scored is itself derived from `roles.go`, so a defect shared between the implementation and the oracle, such as the inherited audit access in Section 7.1, would not appear as a failure here: the campaign confirms the deployed policy is self-consistent, not that it is correct. An independent, requirements-derived permission matrix, reviewed without reference to the implementation and including mutation tests that deliberately alter inheritance edges, would close this gap and is listed as future work in Section 8.3.

These are authorization-boundary tests only. Transport-level attacks, physical tampering and side-channel extraction are not exercised. Mutual TLS and constant-time primitives address them architecturally, not empirically, and Section 8.4 treats them as residual risk rather than as validated defences.

6.7. Transmission energy and cryptographic cost

Reliability of the residue transport is treated in Section 7.2, where it is withdrawn: the deployed encoding sat outside its own recovery bound and the verification path could not have caught the consequence. What the campaign does measure is cost.

Per-event transmission energy falls from 24.20 mJ (SD 0.82) in single-channel mode to 12.41 mJ (SD 0.41) with residue encoding, a reduction of 48.7% [48.6, 48.8], with per-packet airtime falling from 36.1 ms to 31.5 ms. The instrumented packets average 20.5 B and 10.5 B respectively.

Three qualifications, detailed in the supplement, bound what that figure describes: the instrumented payloads are smaller than a complete authenticated frame (an Ed25519 signature alone is 64 B before nonce, timestamp and identity fields), so the percentage is not the saving on deployed traffic; the samples come from one instrumented node, so the interval reflects repeatability rather than between-device variation; and at a 30-minute duty cycle transmission is only 5.8% of a node's daily budget against 9 590 mJ of deep sleep, so halving it moves estimated battery life by about one percent. We report the reduction because it is cleanly measured and make no stronger battery-life claim than that.

Cryptographic cost on the sensor is negligible at this duty cycle. The signing pipeline takes 900 μ s (SD 9), comprising Ed25519 signing at 860 μ s, hardware SHA-256 at 12.0 μ s, eFuse key read at 8.0 μ s and context management at 20.0 μ s. At 0.90 ms per wake against a 30-minute cycle, signing is not a design constraint.

6.8. Availability

The online decision path was available 99.36% of the deployment. 3 connectivity outages occurred, of 8.7 h, 0.2 h and an intermediate duration, totalling 9.3 h, each affecting a single gateway: a LoRa link failure, a power fluctuation and a firmware watchdog reset. In every case the remaining zones kept operating, which is the behaviour the per-zone peer and gateway structure is meant to produce.

Availability figures are only interpretable once the denominator is stated, so we state ours. The indicator is the fraction of deployment hours in which the online authorization path was reachable for *all* zones, which charges a single-gateway outage against the whole system and gives 99.36%. Crediting the zones that kept serving, that is counting gateway-hours, gives 99.84% instead. Both appear in the released results; neither is a statement about request success rates, which we did not record per request.

Neither is a statement about data completeness either. Sensor writes stayed at exactly 2 400 per day on every day including outage days, because gateways buffer readings while disconnected and replay them on recovery. A reading accepted after replay was authorized on replay rather than at capture time, so availability of the decision path and completeness of the record are genuinely different quantities here.

7. Deployment postmortem

Preparing this article we audited the implementation against its own specification. Three defects had run undetected for the entire deployment and one observability gap made a question we wanted to answer unanswerable. We report them together because they share a shape: each is invisible to a benchmark that runs for hours, and each was found by reading code against prose rather than by measurement. Two of the three, the residue transport and the signature path, are corrected in the released artifact; the third, the role hierarchy, is documented but deliberately left as deployed (Section 7.1 explains why), so it is not accurate to say all three are fixed, and we do not say so anywhere else in this article.

7.1. Audit access was inherited upward

The role hierarchy in `roles.go` places Certifier on the human inheritance chain rather than beside it, so Agronomist and Farmer receive `ReadAudit` through inheritance (Equation (1), Table 2). The separation this article's introduction motivates, an auditor who verifies compliance without gaining agronomic access, was therefore never in force. Certifier also holds `ReadZone`, so an auditor could read zone sensor data directly, and Agronomist holds `ControlZone`, which the design denied it.

Three distinct objects are worth separating explicitly, since the rest of this article uses them without restating the distinction each time. *Intended policy*, as Section 3.2 and this article's introduction describe it, denies Agronomist and Farmer audit access and denies Agronomist zone control. *Deployed policy*, what `roles.go` actually enforced for all 61 days, grants both. *Released-artifact policy* is identical to the deployed policy: we have not patched `roles.go` to match the intended design, because doing so would make the released chaincode diverge from the one that produced every measurement in this article, and a reader could no longer check our numbers against the code that generated them. Implementing the intended boundary needs a resource discriminator separating raw sensor state from compliance views, which `CheckAccess` does not have: it matches on permission and zone alone. That change is documented here as a specific, actionable fix for an adopter, not silently applied.

The consequence for the security results is specific. Section 6.6 confirms that the *deployed* policy was self-consistently enforced. It does not validate the intended design, and we make no claim that it does.

7.2. The residue transport was outside its own recovery bound

As Section 4.2 sets out, recovery from an arbitrary pair of residues is unique only below the smallest pairwise product of the moduli. The deployed firmware used $\{97, 101, 103\}$, giving a bound of 9 797, and packed soil moisture and temperature into a single value reaching 823 295. Every reading exceeded the bound by roughly two orders of magnitude, so any reading reassembled from two packets decoded to $v \bmod 9\,797$ rather than to v .

131 795 of the 146 400 committed readings, 90.0%, arrived with two of three residues and were reconstructed by that path. Given the preceding paragraph, that reconstruction is deterministic, not probabilistic: every packed value in this deployment exceeded the 9 797 bound, so every one of those 131 795 reconstructions returned $v \bmod 9\,797$ rather than the original value. All 131 795 two-residue reconstructions in the archive are therefore mathematically incorrect. What we cannot recover is the *original* value each of those readings should have had: the raw residue packets were not retained, the archive stores only the incorrect reconstruction, and we do not claim the corrected value can be recomputed from it after the fact. What the deployment establishes about the redundancy scheme is its cost, not its benefit.

7.3. Signatures were verified over the wrong message

The defect above would have been caught downstream had the signature path been correct. It was not. The firmware signs SHA-256 over a packed binary struct holding device identifier, reading identifier, raw soil moisture, temperature and timestamp. The gateway verified an Ed25519 signature over an ASCII rendering, `device:reading:value:zone`, built from the *decoded* value. These are different messages, so verification could not succeed against a genuine sensor signature. The same routine also returned success whenever a signature was absent, a fail-open default intended to let the gateway run without production credentials.

Taken together, the 100.0% signature-validity figure in the trace cannot mean that sensor signatures were checked and passed. We withdraw the integrity claim rather than qualify it. In the released code the gateway reconstructs the exact bytes the firmware signs, so a mis-decoded reading now fails verification, and unsigned readings are rejected unless a caller opts in explicitly.

7.4. Revocation exposure is not observable

The revocation trace records lifecycle stages with no key linking them into episodes: of 132 subjects, 0 carry all four stages and 87 appear once. End-to-end exposure, the interval from revocation request to enforcement everywhere, is the quantity an operator needs and it cannot be reconstructed. A correlation identifier logged through the pipeline is a small change that would have made it measurable.

7.5. What the pattern suggests

Three of these four went undetected because nothing compared the implementation against its description. The policy diagram and the inheritance table agreed with each other and disagreed with the code; the encoder guarded a bound the producer violated; the verifier and the signer were written against different message formats. Tests existed for each component in isolation and passed.

The artifact now carries checks at those seams: the permission table is generated from `roles.go`, a test asserts the encoder accepts everything the sensor path can produce, and a test pins the gateway's verification

bytes to the firmware's layout. None is sophisticated. Their absence is what allowed a 61-day deployment to run on a system that did not implement its own specification.

What survives this audit for reuse differs by trace. Write-path and peer-scaling latency, throughput, energy and cryptographic timing are performance measurements independent of the three defects above and remain usable as reported. Authorization decisions and denial mechanisms (Section 6.6) are usable as a record of the *deployed* policy, not the intended one. Sensor soil-moisture and temperature values reconstructed from two of three residues, 90.0% of writes, are semantically incorrect and should not be treated as ground truth. The signature-validity field is withdrawn outright. The released data dictionary (`data/raw/README.md`) marks each field with its status so a reader does not have to reconstruct this table from the article.

8. Discussion

8.1. Design implications

Four consequences follow, each contradicting a plausible default assumption.

The recorded authorization-associated span is a quarter of the write path, and the stable quarter. The latency accounting puts it at 319 ms, 26.2% of the write path at 4 peers, and it does not move appreciably when configured peer count is scaled. Effort spent optimising policy evaluation is therefore bounded above by roughly a quarter of write latency, while 397 ms responds to peer provisioning and 503 ms remains in an uninstrumented part of the write path that this design cannot attribute further. A designer choosing where to spend has these three numbers rather than one aggregate, with the caveat that none of the three is a validated Fabric stage. A corollary applies to our own implementation. Because expired-role cleanup mutates state inside `CheckAccess` (Algorithm 1, line 12), read-intent checks are endorsed rather than served as peer queries. Splitting that into a pure read-only decision path, with expiry reclaimed by a background sweep, would take read checks off the consensus path entirely. The measurement identifies this as the highest-value change available, and we did not make it during the deployment precisely because the aggregate latency figure gave no reason to look there.

Peers are associated with lower tail latency in this run; treat throughput separately. Configured peer count is associated with a close-to-log-linear reduction in write latency (Section 6.4), and the saturation point found in the separate, fixed-peer-configuration concurrency experiment sits below the policy layer (Section 6.3). We did not measure throughput as a function of peer count, so we do not extend the "peers buy latency, not capacity" reading to a claim this design tested; a deployment choosing between more endorsers and more capacity should measure both under its own workload rather than assume either result transfers.

Budget access control as a constant. The 9.9% throughput cost showed no trend across a tenfold concurrency range in the fixed-peer-configuration experiment, so capacity planning can treat authorization as a fixed derating factor rather than as a cost that grows with contention, at least at the peer count tested.

Treat revocation as the weak link. It is the only mechanism whose behaviour materially degraded over the deployment, at 29.5% delayed events. The degradation is more consistent with connectivity than with the propagation protocol itself (Section 6.5), but the trace does not isolate the cause, so this is a plausible operational explanation rather than an established one. Mitigations aimed at propagation speed are accordingly unlikely to help; event-driven credential status checking [47] and constrained-device status protocols such as TinyOCSP [8] address a connectivity-dependent failure mode by removing the dependency on a scheduled publication reaching a possibly-disconnected gateway, whichever the cause turns out to be.

8.2. Comparison with reported results

Direct numerical comparison against prior systems is limited by methodological heterogeneity, and saying so is more useful than assembling a misleading table of peak-TPS figures. The systems in Table 1 differ in hardware, workload composition, transaction semantics, durability settings and what is counted as latency. A system reporting a peak rate for a null chaincode on x86 is not measuring the same quantity as this deployment, and Abang et al. [1] make essentially this point about the Fabric latency literature as a whole.

Two comparisons are defensible because they control for platform. Tzenetopoulos et al. [43] report roughly 1.1 s transaction latency for Fabric on Raspberry Pi 3B+ hardware without access-control chaincode. Our

four-peer configuration, which does carry access-control chaincode, sits above that at 1 220 ms, and our 32-peer configuration below it at 821 ms. The crossover is consistent with endorser queueing being the dominant variable at this scale rather than with any claim of superiority. Zaidi et al. [51] report response times near 12 s in simulation at comparable concurrency, an order of magnitude above our field measurements, which we attribute to simulation parameters rather than to a real platform gap and therefore do not claim as an improvement.

The comparison we regard as meaningful is the internal one, the same network, hardware and workload with and without the access-control layer, in Section 6.3. It estimates the cost of the access-control-enabled path against the same platform rather than an isolated permission decision (Section 8.4), and it is reproducible from the released artifact.

8.3. What these measurements cannot establish

Some questions a reader will reasonably ask are outside what this deployment can answer. Setting them out concretely is more useful than a general disclaimer, and each entry names the instrumentation that would settle it.

Whether the residual has a floor. F is the residual at 32 peers, and the measurements do not show latency levelling off there: the per-doubling decrements were still growing. Configurations beyond 32 peers, ideally on physically distinct hosts, would be needed to find where it plateaus and therefore how much of F is genuinely irreducible.

Exact timer boundaries. The trace does not document where the inner access-control counter starts and stops, which is why A and the standalone `CheckAccess` latency are not directly comparable (Section 6.1.1). A reimplementer needs both counters specified at the instruction level, ideally emitted as explicit span boundaries rather than a single duration.

Stage-level attribution inside the residual, and which resource saturates. F stays aggregated over ordering, validation, commit and whatever part of endorsement execution does not respond to peer count, and the 940 ms gap in Section 6.2 is a difference between operation classes and nothing finer. Both arms turning over at 40 clients places the capacity limit below the policy layer without naming it. One transaction timestamped at proposal submission, endorsement completion, orderer receipt, block cut, validation and commit notification, together with orderer queue depth, block-fill rate and CouchDB service times under a controlled intervention, would settle both.

Whether privilege level costs anything. The operational trace's role and operation labels do not separate policy-distinct executions (Section 6.2.1). A designed experiment varying hierarchy depth, cross-zone token presence and denial reason, with the workload otherwise fixed, would answer this.

End-to-end revocation exposure. The trace records lifecycle stages without an episode key, so the interval from revocation request to enforcement everywhere cannot be reconstructed (Section 6.5). Logging a correlation identifier through the pipeline is a small change that would make the operationally important quantity measurable.

Total radio loss and authenticated frame energy. Only committed readings appear in the trace, so transmissions losing two or more residues are invisible and the denominator for a true loss rate is missing. The energy campaign likewise measured the residue-coded payload on one instrumented node. Gateway-side logging of every received residue packet with RSSI and SNR, and an energy campaign over the full authenticated frame on several devices, would close both gaps.

Adversarial security testing, and an oracle independent of the implementation. Section 6.6 is conformance testing against scripted cases, scored against a permission oracle derived from the same `roles.go` the campaign tests, so a defect shared between them would not surface. It says nothing about an attacker who deviates from those scripts. Randomised testing against a defined attack population, and an independent, requirements-derived permission matrix, reviewed without reference to the implementation and exercised with mutation tests that deliberately alter inheritance edges (a mutation removing the auditor separation should make such tests fail, which the present suite cannot), would be a stronger design.

Whether the corrected implementation performs equivalently. The signature and residue-encoding fixes in Section 7 are corrections to code, not to the archived timing measurements, which remain historically valid for the implementation that produced them. We have not run the corrected implementation under the same peer and concurrency configurations, so performance equivalence between the deployed and

corrected code is not established; a laboratory replay at 4 and 32 peers with full authenticated frames would settle whether the corrections change gateway processing time, the recorded authorization span, end-to-end commit latency, throughput, or transmission energy.

8.4. Threats to validity

External. One deployment, one season, one site, with a workload dominated by a regular sensor-write stream. We regard the qualitative finding that the recorded authorization-associated span is a small, stable share of the write path as more transferable than the millisecond values; the peer-scaling finding is qualified further below.

Internal. Overhead is attributed by differencing against a baseline with the access-control chaincode removed, which also removes the audit write, the nonce check and their state access, so the difference bounds the cost of the access-control *layer* rather than of permission evaluation alone. Peer counts above four share physical hardware, so the scaling result varies logical endorsers rather than genuine distribution (Table 4). Each peer-count configuration is a single continuous benchmark run rather than a set of independent trials, and configuration order (4, 8, 16, 32) was not randomized or counterbalanced, so a time-correlated drift, a warming or ledger-growth effect, is a further alternative explanation for the observed latency reduction that this design cannot rule out; repeating the campaign in randomized order across independent runs would separate the two. Throughput was not measured as a function of peer count, so "peers buy latency, not capacity" describes only the latency side of that comparison. Availability is reported against a stated indicator in Section 6.8 and is not a statement about request success rates or data completeness.

Construct. Section 6.6 measures refusals against the deployed policy, which Section 7.1 shows is not the intended one, and its scenario labels do not describe the recorded operations. It is not a conformance result for the designed model. The threat model assumes crash-fault-tolerant ordering with non-malicious orderers.

Data provenance. The released traces are the record of the deployment and the pipeline recomputes every reported value from them. Two caveats are recorded in the dataset documentation: the coordinate columns in the topology trace are unreliable and unused, and the role and operation labels in the operational decision trace do not separate policy-distinct executions (Section 6.2.1).

9. Conclusion

We set out to answer a question that aggregate benchmarks of blockchain access control cannot: how much of the write-path budget is associated with the access-control layer. Across a 61-day deployment recording 209 000 authorization decisions, the write path accounts into 319 ms recorded as an authorization-associated span, 397 ms of remaining latency removable by scaling configured endorsing peers, and a 503 ms remainder at the largest configuration we tested; none of the three is an independently identified Fabric stage. The recorded span is the smallest of the three and the only one shown to hold still under a block-bootstrap comparison: 2.58 ms of difference across an eightfold change in configured peers, well inside a ± 10 ms margin, and a near-constant 9.9% of throughput across a tenfold concurrency range at a fixed peer configuration. Its share of the write path consequently *rises*, from 26.2% at 4 peers to 38.7% at 32, as the peer-sensitive remainder is reduced. Both arms of the separate concurrency experiment saturate at the same offered load, which places the limiting resource below authorization without identifying it.

For an engineer, the consequence is to look past the access-control layer, which is cheap and predictable in this run, toward the uninstrumented remainder of the write path, which is neither. The one component that genuinely degraded over two months was revocation: fast on the happy path at 60.4 s to publish and 4.21 min to propagate, but delayed in 29.5% of 200 lifecycle stage records, a pattern more consistent with connectivity than with the propagation protocol, though the trace does not isolate the cause. That is where the next increment of work belongs, and event-driven credential status checking addresses the observed failure mode rather than an assumed one.

One result supports the model itself: every one of the 8 000 scripted authorization-boundary cases was rejected against the deployed policy, though the test oracle shares its source with the implementation it scores (Section 6.6). Three findings cut the other way, and all three came from auditing the implementation against the manuscript rather than from the measurements. The deployed role hierarchy inherits audit

access upward, so the auditor separation this article motivates was never in force, and we have deliberately left that policy unpatched in the released artifact rather than let the code diverge from the deployment these measurements describe (Section 7.1). The residue transport was mis-parameterised, so every reading recovered from two of three packets decoded incorrectly and the redundancy scheme provided no reliability benefit in the field (Section 7.2). And the gateway verified sensor signatures over a different message than the sensor signed, a fail-open default that would have accepted unsigned readings (Section 7.3). The latter two are fixed in the released code; the corrected implementation's performance has not been separately measured, so we do not claim it reproduces the timings reported here. All three are reasons to treat this as a report on one deployment's observed performance and failure modes, not a validation of the intended security design. The dataset, implementation and the pipeline that regenerates every number here are released so the accounting can be checked, disputed and repeated on other platforms.

CRedit authorship contribution statement

Forcha Glen Beloa: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Writing – original draft. **Michael Ekonde Sone:** Validation, Formal analysis, Supervision, Writing – review & editing. **Elie Fute Tagne:** Resources, Supervision, Project administration, Writing – review & editing. **André Paul Mvogo Asse:** Data curation, Visualization, Writing – review & editing.

Ethics and data protection

The traces record authorization decisions whose subjects include farm operators and administrators, so caller identifiers are potentially personal data. Oversight for the deployment was provided by the University of Buea Faculty of Engineering and Technology's departmental research ethics review, which issued a non-interventional technical study determination (reference FET-CE-2025-014, 14 July 2025) covering the collection of operational, performance and access-control traces from consenting farm personnel; no biomedical or behavioural data were in scope, and no higher-level institutional review was triggered. Each participating operator and administrator gave written informed consent before the deployment began, covering data collection, the use of pseudonymised traces in this article and their public release; participation was voluntary and could be withdrawn without affecting farm operations. Identifiers in the released dataset are pseudonyms assigned at collection (**usr-** and **sns-** prefixes); no names, contact details or free-text fields were collected, and the mapping from pseudonym to individual was destroyed by the data-collection team before the dataset was prepared for release, so no reverse mapping exists for us or for a reader to reconstruct. The site is identified at settlement granularity rather than by field coordinates, and the coordinate columns in the topology trace are unreliable in any case (Section 8.4). No sensor data revealing an individual's conduct beyond routine farm operation was collected. These are the grounds on which the pseudonymised operational traces are released alongside this article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI in the writing process

During the preparation of this work the authors used generative AI tools to assist with language editing and with construction of the analysis pipeline. The authors reviewed and edited all content and take full responsibility for the content of the published article.

Acknowledgements

We thank the participating farm operators in Sfax for their collaboration and practical insight, and acknowledge support from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Programme.

Data availability

The trace dataset, chaincode, firmware, formal specification, benchmark harness and the analysis pipeline that generates every number, table and figure in this article are publicly available at <https://github.com/forchag/blockchain-fet>.

References

- [1] Abang, J.E., Takruri, H., Al-Zaidi, R., Al-Khalidi, M., 2024. Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an IoT perspective. *Internet of Things* 26, 101217. doi:10.1016/j.iot.2024.101217.
- [2] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., Watteyne, T., 2017. Understanding the limits of LoRaWAN. *IEEE Communications Magazine* 55, 34–40. doi:10.1109/MCOM.2017.1600613.
- [3] Alshehri, A., Sandhu, R., 2016. Access control models for cloud-enabled internet of things: A proposed framework, in: *Proceedings of the 2016 IEEE International Conference on Collaboration and Internet Computing (CIC)*, pp. 530–538. doi:10.1109/CIC.2016.075.
- [4] Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., Muralidharan, S., Murthy, C., Nguyen, B., Sethi, M., Singh, G., Smith, K., Sorniotti, A., Stathakopoulou, C., Vukolić, M., Cocco, S.W., Yellick, J., 2018. Hyperledger fabric: A distributed operating system for permissioned blockchains, in: *Proceedings of the Thirteenth EuroSys Conference (EuroSys 2018)*, pp. 30:1–30:15. doi:10.1145/3190508.3190538.
- [5] Bandara, E., Liang, X., De Zoysa, K., Foytik, P., Shetty, S., 2021. Fabric-based access control for agricultural supply-chain traceability: A multi-layer policy model. *IEEE Transactions on Dependable and Secure Computing* 18, 2089–2101. doi:10.1109/TDSC.2021.3092089.
- [6] Benedict, S., 2021. Decentralized access control architecture for IoT-enabled smart environments. *Internet of Things* 14, 100379. doi:10.1016/j.iot.2021.100379.
- [7] Castro, M., Liskov, B., 1999. Practical byzantine fault tolerance, in: *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI 99)*, pp. 173–186.
- [8] Fedrechski, G., Rabaey, J.M., Costa, L.C.P., Calcina-Ccori, P.C., Pereira, W.T., Zuffo, M.K., 2023. TinyOCSP: Lightweight certificate revocation for constrained IoT devices. *Internet of Things* 22, 100740. doi:10.1016/j.iot.2023.100740.
- [9] Ferdous, M.S., Chowdhury, F., Alassafi, M.O., 2019. In search of self-sovereign identity leveraging blockchain technology. *IEEE Access* 7, 103059–103079. doi:10.1109/ACCESS.2019.2931173.
- [10] Ferrag, M.A., Shu, L., Yang, X., Derhab, A., Maglaras, L., 2020. Security and privacy for green IoT-based agriculture: Review, blockchain solutions, and challenges. *IEEE Access* 8, 32031–32053. doi:10.1109/ACCESS.2020.2975715.
- [11] Ferraiolo, D.F., Sandhu, R., Gavrila, S., Kuhn, D.R., Chandramouli, R., 2001. Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security* 4, 224–274. doi:10.1145/501978.501980.
- [12] Garner, H.L., 1959. The residue number system. *IRE Transactions on Electronic Computers* EC-8, 140–147. doi:10.1109/TEC.1959.5219515.
- [13] González, V., Sánchez, L., Lanza, J., Santana, J.R., Sotres, P., García, A.E., 2023. On the use of blockchain to enable a highly scalable Internet of Things data marketplace. *Internet of Things* 22, 100722. doi:10.1016/j.iot.2023.100722.
- [14] Hernández-Ramos, J.L., Jara, A.J., Marín, L., Skarmeta, A.F., 2013. Distributed capability-based access control for the internet of things. *Journal of Internet Services and Information Security* 3, 1–16.
- [15] Hu, V.C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K., 2014. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. Technical Report NIST SP 800-162. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-162.
- [16] Hyperledger Foundation, 2025. Hyperledger fabric v3.0: What's new. Available at <https://hyperledger-fabric.readthedocs.io/en/release-3.0/whatsnew.html>.
- [17] Idrissi, M., Palmieri, F., 2024. Blockchain-based authentication and authorization for IoT systems: An end-to-end latency analysis on constrained hardware. *The Journal of Supercomputing* 80, 1–32. doi:10.1007/s11227-024-06001-7.
- [18] Kamilaris, A., Fonts, A., Prenafeta-Boldú, F.X., 2019. The rise of blockchain technology in agriculture and food supply chains. *Trends in Food Science & Technology* 91, 640–652. doi:10.1016/j.tifs.2019.07.034.
- [19] Ke, C.K., Park, S., 2023. Performance modeling of hyperledger fabric for IoT access control applications. *Future Internet* 15, 115. doi:10.3390/fi15030115.
- [20] Lamport, L., 2002. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley.
- [21] Lea-Cox, J.D., Belayneh, A., van Iersel, M.W., Lichtenberg, E., Kantor, G.F., 2013. Low-cost soil moisture sensor networks for precision irrigation: Advances and limitations. *HortTechnology* 23, 792–798. doi:10.21273/HORTTECH.23.6.792.
- [22] Lin, Q., Wang, H., Pei, X., Wang, J., 2019. Food safety traceability system based on blockchain and EPCIS. *IEEE Access* 7, 20698–20707. doi:10.1109/ACCESS.2019.2897792.
- [23] Liu, Y., Yu, F.R., Li, X., Ji, H., Leung, V.C.M., 2019. Distributed resource allocation in blockchain-based video streaming systems with mobile edge computing. *IEEE Transactions on Wireless Communications* 18, 695–708. doi:10.1109/TWC.2018.2886345.

- [24] Mohanta, B.K., Awad, A.I., Elsaka, T., Kheddar, H., Baraka, E., 2025. Smart-contract-based blockchain-enabled decentralized scheme for improving smart-grid security. *Internet of Things* 34, 101811. doi:10.1016/j.iot.2025.101811.
- [25] Neumann, A., Aichele, C., Lindner, M., Wunderlich, S., 2008. Better Approach To Mobile Ad-hoc Networking (B.A.T.M.A.N.). Internet Draft draft-openmesh-b-a-t-m-a-n-00. IETF. Available at <https://tools.ietf.org/html/draft-openmesh-b-a-t-m-a-n-00>.
- [26] Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M., 2015. How Amazon Web Services uses formal methods. *Communications of the ACM* 58, 66–73. doi:10.1145/2699417.
- [27] Novo, O., 2018. Blockchain meets IoT: An architecture for scalable access management in IoT. *IEEE Internet of Things Journal* 5, 1184–1195. doi:10.1109/JIOT.2018.2812239.
- [28] Ongaro, D., Ousterhout, J., 2014. In search of an understandable consensus algorithm, in: *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319.
- [29] Ouaddah, A., Abou Elkalam, A., Ait Ouahman, A., 2016. FairAccess: A new blockchain-based access control framework for the Internet of Things. *Security and Communication Networks* 9, 5943–5964. doi:10.1002/sec.1748.
- [30] Pajooh, H.H., Hammoudeh, M., Alam, F., Brocklehurst, S., 2021. Hyperledger fabric blockchain for securing the edge IoT: Systems architecture, experiments and critical observations. *Electronics* 10, 1338. doi:10.3390/electronics10111338.
- [31] Putra, G.D., Dedeoglu, V., Kanhere, S.S., Jurdak, R., 2021. Trust management in decentralized IoT access control system, in: *Proc. IEEE BCCA*, pp. 1–8. doi:10.1109/BCCA53669.2021.9657049.
- [32] Raza, S., Shafagh, H., Hewage, K., Hummen, R., Voigt, T., 2013. Lithe: Lightweight secure CoAP for the Internet of Things. *IEEE Sensors Journal* 13, 3711–3720. doi:10.1109/JSEN.2013.2277656.
- [33] Rustemi, A., Dalipi, F., 2025. Synergizing IoT, AI, and blockchain for smart agriculture: Challenges, opportunities, and future directions. *Internet of Things* 34, 101778. doi:10.1016/j.iot.2025.101778.
- [34] Safeer, S., Gallo, P., Pulvento, C., 2025. Agri-farming with computer vision, IoT and blockchain towards climate smart cultivation. *Internet of Things* 34, 101749. doi:10.1016/j.iot.2025.101749.
- [35] Sandhu, R., Bhamidipati, V., Munawar, Q., 1999. The ARBAC97 model for role-based administration of roles. *ACM Transactions on Information and System Security* 2, 105–135. doi:10.1145/300830.300839.
- [36] Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E., 1996. Role-based access control models. *Computer* 29, 38–47. doi:10.1109/2.485845.
- [37] Sciancalepore, S., Piro, G., Boggia, G., Bianchi, G., 2017. OAuth-IoT: An access control framework for the Internet of Things based on open standards, in: *Proc. IEEE ISCC*, pp. 676–681. doi:10.1109/ISCC.2017.8024606.
- [38] Servos, D., Osborn, S.L., 2017. Current research and open problems in attribute-based access control. *ACM Computing Surveys* 49, 65:1–65:45. doi:10.1145/2983429.
- [39] Shih, D.H., Jheng, P.Y., Yen, D.C., Huang, C.L., 2022. A blockchain-based access control model for industrial IoT. *Sensors* 22, 2188. doi:10.3390/s22062188.
- [40] Tara, S., Zeadally, S., Sookun, D., 2023. Decentralized identity management for IoT devices using blockchain and zero-knowledge proofs. *IEEE Internet of Things Journal* 10, 3817–3827. doi:10.1109/JIOT.2022.3211254.
- [41] Thakkar, P., Nathan, S., Viswanathan, B., 2018. Performance benchmarking and optimizing hyperledger fabric blockchain platform, in: *Proceedings of the 26th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pp. 264–276. doi:10.1109/MASCOTS.2018.00060.
- [42] Tian, F., 2017. A supply chain traceability system for food safety based on HACCP, blockchain and internet of things, in: *Proceedings of the 14th International Conference on Service Systems and Service Management (ICSSSM)*, pp. 1–6. doi:10.1109/ICSSSM.2017.7996119.
- [43] Tzenetopoulos, A., Katsaros, P., Mavridis, I., 2022. HLF-Kubed: Hyperledger fabric blockchain on edge clusters for IoT identity and access control. *Future Generation Computer Systems* 135, 271–283. doi:10.1016/j.future.2022.05.003.
- [44] Usman, M., Jolfaei, A., Tariq, M.U., 2024. Blockchain-based access control for industrial IoT: Domain policy architecture with smart contracts. *Computer Networks* 243, 110286. doi:10.1016/j.comnet.2024.110286.
- [45] Wang, X., Han, Y., Leung, V.C.M., Niyato, D., Yan, X., Chen, X., 2020. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 22, 869–904. doi:10.1109/COMST.2020.2970550.
- [46] World Wide Web Consortium, 2022. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, 19 July 2022. Available at <https://www.w3.org/TR/did-core/>.
- [47] World Wide Web Consortium, 2024. Bitstring status list v1.0. W3C Candidate Recommendation Snapshot, 5 March 2024. Available at <https://www.w3.org/TR/vc-bitstring-status-list/>.
- [48] Xu, R., Chen, Y., Blasch, E., Chen, G., 2018. BlendCAC: A blockchain-enabled decentralized capability-based access control for IoTs, in: *Proceedings of the 2018 IEEE International Conference on Internet of Things (iThings)*, pp. 1027–1034. doi:10.1109/Cybermatics_2018.2018.00191.
- [49] Ye, N., Zhu, Y., Wang, R.c., Malekian, R., Lin, Q.m., 2014. An efficient authentication and access control scheme for perception layer of internet of things. *Applied Mathematics & Information Sciences* 8, 1617–1624. doi:10.12785/amis/080416.
- [50] Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I., 2019. HotStuff: BFT consensus with linearity and responsiveness, in: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 347–356. doi:10.1145/3293611.3331591.
- [51] Zaidi, S.M.S., Al-Qurishi, M., Abidin, M.Z.u., Almakdi, S., Asiri, Y., 2023. Fabrication: A hyperledger fabric framework for scalable IoT access control. *IEEE Internet of Things Journal* 10, 12636–12649. doi:10.1109/JIOT.2023.3254978.

- [52] Zhang, N., Wang, M., Wang, N., 2002. Precision agriculture: A worldwide overview. *Computers and Electronics in Agriculture* 36, 113–132. doi:10.1016/S0168-1699(02)00056-X.